Restaurant Management System

Project Title: Restaurant Management System.
Course: CSC 478 – Software Engineering Capstone
Team Name: Error 404
Team Members: Haripriya Koneru, Krishna Kumar Nimmagadda, Venkata Sai Gowtham Panyam, Vansh Parihar, Brijeshkumar Mansukhbhai Raiyani, Pratham Patel, Vijaya Venkateswara Rao Rowthu
Document: Design Documentation
Frontend Repository: https://github.com/Patel-Pratham-3492/Error_404
Backend Repository: https://github.com/Patel-Pratham-3492/Error_404_server

Design Documentation

This document describes the design of the Restaurant Management System (RMS), a full-stack web application developed to streamline daily restaurant operations. The RMS provides a centralized, role-based interface for managing menu items, processing customer orders, and coordinating activities between waiters, chefs, managers, and hosts.

The system is structured as a React-based frontend and a Node.js/Express backend connected to MongoDB for persistent storage. Frontend deployment is handled through Render, while the backend runs locally or via Node server hosting without AWS. Authentication is implemented using **session tokens** rather than JWT. The RMS is designed to provide a flexible and extensible foundation that can grow into a production-grade restaurant solution.

1.1 Architecture Diagram

The overall system follows a **client–server architecture** with a RESTful API layer.
The frontend is responsible for rendering the user interface and sending requests to the backend, while the backend handles business logic, session-based authentication, and interaction with MongoDB.

Frontend Architecture

The frontend is implemented using **React with Vite**, structured around reusable components and role-based page layouts. Axios is used for communication with backend APIs. State is managed locally within components or via context for shared state, such as logged-in user information.

Backend Architecture

The backend is a **Node.js application using Express**. It exposes various endpoints to support authentication, menu management, and order operations. The backend maintains session-based authentication using server-stored session tokens rather than JWT. Each session is mapped to a specific user and role.

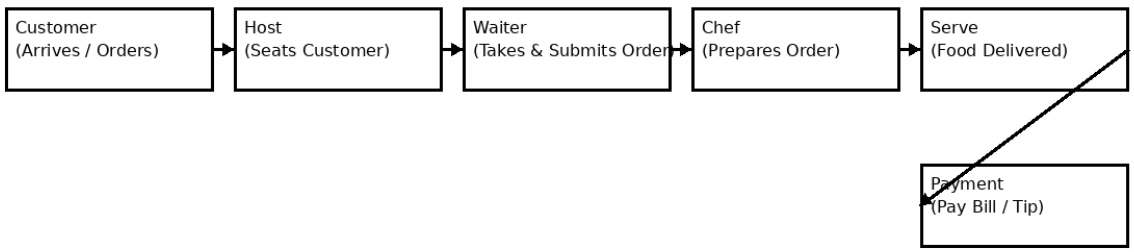
MongoDB, accessed through Mongoose, stores all persistent data: users, orders, and menu items.

Authentication Design

Unlike token-based systems using JWT, RMS uses **session-based login**.
When a user logs in through the `api/login` API endpoint:

1. The backend verifies credentials.
2. If valid, a server-side session is created and linked to the user.
3. A session token is stored in an HTTP-only cookie on the client.
4. The cookie is attached automatically in each subsequent request.
5. Session validity is checked before allowing access to protected routes.

This approach allows tight control on the server side and simplifies invalidation and logout.



The RMS allows different types of restaurant staff to perform their tasks through tailored dashboards:

- **Waiters** enter and update customer orders.
- **Chefs** view incoming orders and update preparation status.
- **Managers** control menu items and staff configurations.
- **Hosts** assign tables and manage seating.
- **Owners** oversee restaurant performance at a high level.

The system supports a multi-step ordering workflow where customers may place starters, main courses, and desserts at different times. All order changes are reflected in real time through API communication between the frontend and backend.

1.2 System Overview:

The RMS is a full-stack web application designed to help restaurants manage menus, orders, and user roles (Manager, Chef, Waiter, Host, Owner). The system provides:

- Menu management
- Order creation and tracking
- Role-based access and dashboards
- Status updates for preparation
- Authentication (JWT)
- Deployment on Render (frontend) and AWS (backend)

MongoDB provides persistent storage for all menu items, users, orders, and roles.

UI responsibilities

- Present role-specific views: Host dashboard, Waiter POS, Kitchen board, Service runner view, Payment/receipt screen.
- Manage local UI state and transient states (submitting, optimistic updates, offline cache).
- Send user actions to backend (create order, change status, request bill) and listen for realtime events to update views.
- Provide clear feedback for long operations, failures, and retries.
- Enforce client-side validations and friendly error messaging (but not security-critical enforcement).

Backend responsibilities

- Provide authenticated REST endpoints that implement the business operations (orders, tables, menu, payments).
- Enforce authorization and role-based rules for every action.

- Validate and normalize incoming data, perform business logic (pricing, modifiers, inventory checks).
- Persist authoritative state in MongoDB and emit events for realtime updates.
- Integrate with external systems (payment processors, analytics) and manage webhooks/callbacks.
- Provide observability (metrics, logs, traces) and durable audit trails for each state change.

Authentication & session flow

- Authentication is session-based: a successful login creates a server-side session, and the browser sends an authentication cookie on subsequent requests.
- The server stores session metadata (user id, role, TTL) in a persistent session store so multiple Express instances can validate sessions.
- Every API endpoint checks the session for authentication and role authorization; the UI obtains the current user/role from the backend rather than trusting client-side state.
- Because sessions use cookies, protect against CSRF and session fixation and set cookie attributes (secure, httpOnly, sameSite) according to deployment.

Data model

- Core entities: Users (staff roles), Tables, Customers (optional), Menu Items, Orders, Order Items (embedded in orders), Payments, Audit logs.
- Orders are the authoritative transaction that tie table + staff + items + status + payments.
- Order items are often modeled as embedded subdocuments inside an order for atomicity of order updates.
- Use indices for common queries: status, createdAt, table, staff, idempotency keys.

API design principles

- Use RESTful resources representing business entities (orders, tables, menu).
- Keep endpoints coarse-grained enough to align with business actions (e.g., “create order”, “update order status”, “submit payment”).
- Require an idempotency token for non-idempotent operations clients might retry (order creation) so the server can deduplicate.
- Always return canonical server state after write operations so the client can reconcile optimistic UI state.
- Design payloads to include minimal but sufficient metadata so clients can update local caches without repeated full-state fetches.

Real-time interaction

- Use a real-time channel (WebSocket/Socket.IO or equivalent) to publish domain events: order.created, order.updated, order.ready, order.served, payment.completed.
- Clients subscribe by role and interest (e.g., kitchen subscribes to kitchen events, a waiter subscribes to the table/party).

- Events contain enough metadata (IDs, status, timestamps) so clients can update local view and optionally fetch additional details.
- For scale, real-time pub/sub should be backed by a shared broker/adapter so multiple server instances can publish/subscribe.

Ordering lifecycle and consistency

- Order lifecycle states: received → preparing→ served (Done).
- Backend is authoritative about transitions and enforces valid state changes (e.g., cannot mark served unless ready).
- Favor single-document updates for order modifications to leverage MongoDB atomicity. For multi-document workflows (e.g., mark table occupied + create order), use transactions (replica-set transactions) or strong application-level coordination.
- Use optimistic UI on client but always reconcile with server response and handle conflicts (show user conflict messages or fetch fresh state).